

Análise Empírica de Dívida Técnica e Manutenibilidade em Frameworks e Bibliotecas Java Open-Source: Um Estudo Comparativo Profundas

Guia Prático de Replicação do Experimento – Projeto Mockito

Discentes: André de Oliveira Cacau, Paulo Ricardo Oliveira de Aguiar

Docente: Prof^ª. Dr^ª. Catarina de Souza Costa

Mestrado em Ciência da Computação – Engenharia de Software
Universidade Federal do Acre (UFAC)

1

Abstract. *This document provides a detailed step-by-step guide to reproducing the empirical analysis environment and static code extraction for the Mockito project. It covers containerized setup using Docker, SonarQube Community Edition, analysis execution via SonarScanner CLI, and automated metrics extraction via Python and REST API.*

Resumo. *Este documento apresenta o passo a passo detalhado para a reprodução do ambiente de análise empírica e extração estática de código do projeto Mockito. Abrange desde a infraestrutura em contêineres Docker com SonarQube Community Edition, passando pela execução da análise via SonarScanner CLI, até a extração automatizada das métricas via Python e API REST.*

1. Introdução e Pré-requisitos

Este relatório descreve o procedimento prático para executar a análise de qualidade e dívida técnica na biblioteca de metaprogramação e mocks **Mockito**. O ambiente foi projetado para ser totalmente isolado e reproduzível por meio de contêineres Docker.

1.1. Pré-requisitos do Sistema

- **Docker Desktop** instalado e em execução (com suporte a Docker Compose).
- **Git** para clonagem do repositório.
- **Python 3.8+** para execução do script de extração.
- Biblioteca `requests` do Python (`pip install requests`).

2. Estrutura de Diretórios Recomendada

Para garantir a organização dos artefatos do experimento, recomenda-se estruturar os diretórios da seguinte forma:

```
1 experimento-sq/  
2 |-- docker-compose.yml  
3 |-- extrair_metricas_mockito.py  
4 |-- repos/  
5 |-- mockito/
```

Listing 1. Estrutura de diretórios do experimento

3. Configuração do Ambiente com Docker

3.1. Criando o arquivo `docker-compose.yml`

Caso ainda não tenha o ambiente rodando, crie o arquivo `docker-compose.yml` na pasta `experimento-sq/`:

```
1 version: '3.8'
2
3 services:
4   sonarqube:
5     image: sonarqube:community
6     container_name: sonarqube_exp
7     ports:
8       - "9000:9000"
9     environment:
10      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
11     volumes:
12       - sonarqube_data:/opt/sonarqube/data
13       - sonarqube_extensions:/opt/sonarqube/extensions
14       - sonarqube_logs:/opt/sonarqube/logs
15
16 volumes:
17   sonarqube_data:
18   sonarqube_extensions:
19   sonarqube_logs:
```

Listing 2. Arquivo `docker-compose.yml`

3.2. Iniciando o Serviço

No terminal, dentro da pasta `experimento-sq/`, suba o contêiner:

```
1 docker compose up -d
```

Acesse o painel web no endereço `http://localhost:9000` (login/senha inicial: `admin / admin`). Em **Account & Security**, gere um ****Token de Acesso**** para autenticação nos passos seguintes.

4. Clonagem e Varredura do Mockito

4.1. Clonando o Repositório

Acesse o diretório `repos/` e clone o repositório oficial do Mockito:

```
1 cd repos/
2 git clone https://github.com/mockito/mockito.git
3 cd mockito
```

4.2. Executando a Análise Estática com SonarScanner

A partir do diretório `repos/mockito`, execute a varredura via contêiner do SonarScanner:

```
1 docker run --rm \
2   --network="host" \
3   -v "$(pwd) :/usr/src" \
4   sonarsource/sonar-scanner-cli \
5   -Dsonar.projectKey=mockito \
```

```

6 -Dsonar.projectName="Mockito" \
7 -Dsonar.sources=. \
8 -Dsonar.host.url=http://localhost:9000 \
9 -Dsonar.token=SEU_TOKEN_AQUI \
10 -Dsonar.java.binaries=.

```

Nota: Substitua SEU_TOKEN_AQUI pelo token gerado no painel do SonarQube.

5. Extração Automatizada dos Dados via API REST

Após o término da execução do SonarScanner, utilize o script Python para extrair as métricas de manutenibilidade de cada arquivo .java.

5.1. Script de Extração (extrair_metricas_mockito.py)

Crie o arquivo extrair_metricas_mockito.py na raiz do projeto com o seguinte código:

```

1 import requests
2 import csv
3
4 SONAR_URL = "http://localhost:9000"
5 PROJECT_KEY = "mockito"
6 TOKEN = "SEU_TOKEN_AQUI" # Insira seu token do SonarQube
7
8 METRICS = [
9     "cognitive_complexity",
10    "sqale_index",          # Debito Tecnico em minutos
11    "sqale_debt_ratio",    # TDR
12    "code_smells",
13    "bugs",
14    "vulnerabilities",
15    "ncloc"                # Linhas de codigo
16 ]
17
18 def fetch_measures():
19     url = f"{SONAR_URL}/api/measures/component_tree"
20     params = {
21         "component": PROJECT_KEY,
22         "metricKeys": ",".join(METRICS),
23         "ps": 500,
24         "p": 1,
25         "qualifiers": "FIL"
26     }
27
28     headers = {"Authorization": f"Bearer {TOKEN}"}
29     all_components = []
30
31     while True:
32         res = requests.get(url, params=params, headers=headers)
33         if res.status_code != 200:
34             print(f"Erro na requisicao: {res.status_code}")
35             break
36
37         data = res.json()
38         components = data.get("components", [])

```

```

39     all_components.extend(components)
40
41     paging = data.get("paging", {})
42     if paging.get("pageIndex", 1) * paging.get("pageSize", 500) >=
43         paging.get("total", 0):
44         break
45     params["p"] += 1
46
47     return all_components
48
49 def save_to_csv(components, filename="mockito_metrics.csv"):
50     with open(filename, mode="w", newline="", encoding="utf-8") as f:
51         writer = csv.writer(f)
52         header = ["path", "language"] + METRICS
53         writer.writerow(header)
54
55         for comp in components:
56             if not comp.get("path", "").endswith(".java"):
57                 continue
58
59             measures = {m["metric"]: m.get("value", "0") for m in comp.
60 get("measures", [])}
61             row = [
62                 comp.get("path"),
63                 comp.get("language")
64             ] + [measures.get(m, "0") for m in METRICS]
65
66             writer.writerow(row)
67
68             print(f"Dados salvos com sucesso em {filename}!")
69
70 if __name__ == "__main__":
71     print("Iniciando extracao de metricas do Mockito...")
72     data = fetch_measures()
73     save_to_csv(data)

```

Listing 3. Script de extração via API REST para Mockito

5.2. Executando o Script

Execute o script no terminal:

```
python extrair_metricas_mockito.py
```

O arquivo `mockito_metrics.csv` será gerado e estará pronto para ser incorporado às etapas de análise estatística e consolidação da matriz de criticidade do estudo.

6. Considerações Finais

Com este relatório, completa-se o conjunto de guias de replicação para os três ecossistemas estudados (Spring Boot, JUnit 5 e Mockito). Toda a infraestrutura e código-fonte aqui descritos asseguram a reprodutibilidade integral dos resultados científicos alcançados.